

CyberAudit

Authentication Flow

Register · Login · Logout · JWT · Auto-refresh · localStorage

Project / Projet	CyberAudit Platform
Frontend	React 19 + Vite
Backend	Django 5.1 + DRF + SimpleJWT
Auth method	JWT : access + refresh tokens
Session storage	localStorage key: "cyberaudit:session"
Access token TTL	60 minutes
Refresh token TTL	60 days (ROTATE_REFRESH_TOKENS=True)
Source files	src/services/api.js src/services/authService.js src/context/AuthContext.jsx src/hooks/useAuth.js
Date	June 2026 / Juin 2026

1. Architecture Overview

1. Vue d'ensemble de l'architecture

The authentication flow is split across four files. Each has a single responsibility: `authService.js` calls the API, `api.js` manages the Axios interceptors, `AuthContext.jsx` holds the global React state, and `useAuth.js` exposes the context via a clean hook.

Le flux d'authentification est réparti sur quatre fichiers. Chacun a une responsabilité unique : `authService.js` appelle l'API, `api.js` gère les intercepteurs Axios, `AuthContext.jsx` maintient l'état React global, et `useAuth.js` expose le contexte via un hook simple.

File / Fichier	Layer / Couche	Responsibility / Responsabilité
api.js	HTTP client	(EN) Axios instance + request interceptor (Bearer token injection) + response interceptor (401 auto-refresh) / (FR) Instance Axios + intercepteur requête (injection Bearer) + intercepteur réponse (auto-refresh 401)
authService.js	API calls	(EN) register / login / logout / getSession / getMe / changePassword each maps to a real Django endpoint / (FR) Chaque fonction correspond à un endpoint Django réel
AuthContext.jsx	React state	(EN) Global auth state: user, token, ready, isAuthenticated, hasRole restored from localStorage on mount / (FR) État global : user, token, ready, isAuthenticated restauré depuis localStorage au montage
useAuth.js	Custom hook	(EN) Thin wrapper around useContext(AuthContext) used in every component that needs auth / (FR) Wrapper léger autour de useContext(AuthContext) utilisé dans chaque composant qui a besoin de l'auth

Session data format / Format des données de session

All four files share the same localStorage key and data shape. The key is a namespaced constant so it never conflicts with other apps.

Les quatre fichiers partagent la même clé localStorage et la même structure de données. La clé est une constante avec namespace pour éviter les conflits.

```
// localStorage key (shared between api.js and authService.js)
const SESSION_KEY = "cyberaudit:session";

// Value stored (JSON-stringified):
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTUwMn0", // JWT access token (60-min lifetime)
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTUwMn0", // JWT refresh token (60-day lifetime)
  "user": {
    "id": "uuid-...",
    "email": "user@example.com",
  }
}
```

```
"first_name": "Tommy",  
"last_name": "JOUHANS",  
"company_name": "Acme Corp",  
"role": "client"  
}  
}
```

2. Register

2. Inscription

A new user fills in the registration form (email, password, first name, last name, company). The frontend calls POST /api/auth/register/ on the Django backend. On success, Django returns access and refresh tokens + a user object, which are saved to localStorage immediately, the user is logged in right after registering.

Un nouvel utilisateur remplit le formulaire d'inscription (email, mot de passe, prénom, nom, société). Le frontend appelle POST /api/auth/register/ sur le backend Django. En cas de succès, Django retourne les tokens access et refresh + un objet user, qui sont immédiatement sauvegardés en localStorage, l'utilisateur est connecté juste après l'inscription.

Endpoint / API endpoint

Method	POST
URL	/api/auth/register/
Auth required?	(EN) No : public endpoint / (FR) Non : endpoint public
Request body	email, password, first_name, last_name, company_name
Response (200)	{ access, refresh, user: { id, email, first_name, last_name, company_name, role } }
On error	{ email: ['already exists'] } or { non_field_errors: [...] } etc. { email: ['existe déjà'] } ou { non_field_errors: [...] } etc.

Source code : authService.js

```
// authService.js - register()
export async function register(form) {
  try {
    const { data } = await api.post("/auth/register/", {
      email: form.email?.trim(),
      password: form.password,
      first_name: (form.firstName ?? form.first_name ?? "").trim(),
      last_name: (form.lastName ?? form.last_name ?? "").trim(),
      company_name: (form.companyName ?? form.company_name ?? "").trim(),
    });

    // Save access + refresh + user to localStorage
    return saveSession(data.access, data.refresh, data.user);
  } catch (error) {
    throw new Error(extractErrorMessage(error), { cause: error });
  }
}

// AuthContext.jsx - wraps the service call, updates React state
const register = useCallback(async (form) => {
  const session = await authService.register(form);
  setUser(session.user);
}, []);
```

```
setToken(session.access);  
return session;  
, []);
```

Step-by-step flow / Flux étape par étape

1. User submits form	React component calls: <code>const { register } = useAuth(); await register(formData);</code>
2. <code>authService.register()</code>	POST <code>/api/auth/register/</code> with email, password, first_name, last_name, company_name
3. camelCase → snake_case	<code>form.firstName</code> ?? <code>form.first_name</code> (EN) accepts both naming conventions from the form / (FR) Accepte les deux conventions de nommage du formulaire.
4. <code>saveSession()</code>	<code>localStorage.setItem('cyberaudit:session', JSON.stringify({ access, refresh, user }))</code>
5. AuthContext updates	<code>setUser(session.user) + setToken(session.access)</code> (EN) React state updated globally / (FR) État React mis à jour globalement.
6. User is now logged in	(EN) <code>isAuthenticated = true</code> . Protected routes become accessible. (FR) <code>isAuthenticated = true</code> . Les routes protégées deviennent accessibles.

Error handling / Gestion des erreurs

The `extractErrorMessage()` helper normalises Django REST Framework error responses into a single readable string for the UI. DRF can return errors in three formats depending on the type of validation failure.

L'utilitaire `extractErrorMessage()` normalise les réponses d'erreur de Django REST Framework en une seule chaîne lisible pour l'interface. DRF peut retourner des erreurs dans trois formats selon le type d'échec de validation.

```
// authService.js - extractErrorMessage()  
function extractErrorMessage(error) {  
  const data = error.response?.data;  
  if (!data) return error.message || "Network error."  
  
  if (data.non_field_errors) return data.non_field_errors[0]; // e.g. wrong  
  password  
  if (data.detail) return data.detail; // e.g. "No active  
  account found"  
  
  // First field-level error: e.g. { email: ["already exists"] }  
  const firstKey = Object.keys(data)[0];  
  if (firstKey) {  
    const msg = data[firstKey];  
    return Array.isArray(msg) ? msg[0] : msg;  
  }  
  return "An error has occurred."  
}
```

3. Login

3. Connexion

The user provides their email and password. The frontend calls POST /api/auth/login/. The email is trimmed and lower-cased before sending. On success, the session is saved to localStorage and the React state is updated.

L'utilisateur fournit son email et son mot de passe. Le frontend appelle POST /api/auth/login/. L'email est nettoyé et mis en minuscules avant l'envoi. En cas de succès, la session est sauvegardée en localStorage et l'état React est mis à jour.

Endpoint / API endpoint

Method	POST
URL	/api/auth/login/
Auth required?	(EN) No : public endpoint / (FR) Non : endpoint public
Request body	{ "email": "user@example.com", "password": "secret" }
Response (200)	{ access, refresh, user: { id, email, first_name, last_name, company_name, role } }
On error (401)	{ "detail": "No active account found with the given credentials." }

Source code : authService.js

```
// authService.js - login()
export async function login(email, password) {
  try {
    const { data } = await api.post("/auth/login/", {
      email: email.trim().toLowerCase(), // normalise before sending
      password,
    });

    // Save tokens + user to localStorage
    return saveSession(data.access, data.refresh, data.user);
  } catch (error) {
    throw new Error(extractErrorMessage(error), { cause: error });
  }
}

// Helper: saves { access, refresh, user } to localStorage
function saveSession(access, refresh, user) {
  const session = { access, refresh, user };
  localStorage.setItem(SESSION_KEY, JSON.stringify(session));
  return session;
}
```

Step-by-step flow / Flux étape par étape

1. User submits form	const { login } = useAuth(); await login(email, password);
2. Email normalisation	email.trim().toLowerCase() (EN) prevents case-sensitivity issues.(FR) Évite les problèmes de casse.
3. POST /auth/login/	Django validates credentials against the custom User model (UUID + email-based auth).
4. On success	saveSession(access, refresh, user) → localStorage.setItem(SESSION_KEY, JSON.stringify({...}))
5. React state update	setUser(session.user) + setToken(session.access)
6. Navigation	Component redirects to dashboard: navigate('/dashboard')

Session restoration at startup / Restauration de session au démarrage

When the app loads, AuthContext reads localStorage before rendering any route. This prevents the protected route from flashing a redirect to /login when the user is actually logged in.

Au chargement de l'app, AuthContext lit localStorage avant de rendre n'importe quelle route. Cela empêche la route protégée d'afficher brièvement une redirection vers /login alors que l'utilisateur est bien connecté.

```
// AuthContext.jsx — session restoration on mount
useEffect(() => {
  const session = authService.getSession(); // reads localStorage
  if (session) {
    setUser(session.user);
    setToken(session.access);
  }
  setReady(true); // signals: session check done, safe to render routes
}, []);

// getSession() in authService.js
export function getSession() {
  const raw = localStorage.getItem(SESSION_KEY);
  if (!raw) return null;
  const session = JSON.parse(raw);
  // Migration: old mock format { token, user } — clear it, force re-login
  if (session?.token && !session?.access) {
    localStorage.removeItem(SESSION_KEY);
    return null;
  }
  return session;
}

// ready flag prevents route flicker:
// if (!ready) return <Spinner />; // hold routes until session is known
```

4. Logout

4. Déconnexion

Logout is a two-step operation: (1) send the refresh token to the server to blacklist it (so it cannot be used to generate new access tokens even if stolen), (2) remove the session from localStorage and reset the React state.

La déconnexion est une opération en deux étapes : (1) envoyer le refresh token au serveur pour le blacklister (afin qu'il ne puisse pas générer de nouveaux access tokens même s'il est volé), (2) supprimer la session du localStorage et réinitialiser l'état React.

Endpoint / API endpoint

Method	POST
URL	/api/auth/logout/
Auth required?	(EN) Yes, sends Bearer access token in Authorization header / (FR) Oui, envoie le Bearer access token dans l'en-tête Authorization
Request body	{ "refresh": "<refresh_token>" }
Response (200)	(FR) Empty {} (token is now blacklisted on the server) (FR) Vide {} (le token est maintenant blacklisté côté serveur)
Best-effort	(EN) Even if the server call fails, the local session is always cleared. / (FR) Même si l'appel serveur échoue, la session locale est toujours effacée.

Source code — authService.js

```
// authService.js - logout()
export async function logout() {
  try {
    const raw = localStorage.getItem(SESSION_KEY);
    if (raw) {
      const session = JSON.parse(raw);
      if (session?.refresh) {
        // Tell server to blacklist this refresh token (best-effort)
        await api.post("/auth/logout/", { refresh: session.refresh });
      }
    }
  } catch {
    // Server call failed - that is OK, we still clear locally
  } finally {
    // Always remove the session regardless of server response
    localStorage.removeItem(SESSION_KEY);
  }
}

// AuthContext.jsx - clears React state after service call
const logout = useCallback(async () => {
  await authService.logout();
}, []);
```



```
setUser(null);  
setToken(null);  
}, []);
```

Why blacklist the refresh token? / Pourquoi blacklister le refresh token ?

Without server-side blacklisting, a stolen refresh token would remain valid for 60 days. Django SimpleJWT's `token_blacklist` app invalidates it immediately on the server. Even if an attacker intercepts the token before logout, it becomes unusable the moment the user logs out.

Sans blacklistage côté serveur, un refresh token volé resterait valide 60 jours. L'app `token_blacklist` de Django SimpleJWT l'invalidé immédiatement sur le serveur. Même si un attaquant intercepte le token avant la déconnexion, il devient inutilisable dès que l'utilisateur se déconnecte.

ROTATE_REFRESH_TOKENS=True	(EN) Each <code>/token/refresh/</code> call issues a new refresh token AND blacklists the old one. Limits replay attacks / (FR) Chaque appel <code>/token/refresh/</code> émet un nouveau refresh token ET blackliste l'ancien. Limite les attaques par rejeu.
BLACKLIST_AFTER_ROTATION=True	(EN) Enforces that rotated-out refresh tokens are always blacklisted immediately. (FR) Force le blacklistage immédiat des refresh tokens tournés.
token_blacklist app	django.contrib.apps: rest_framework_simplejwt.token_blacklist. (EN) Stores invalidated JTIs in the DB. (FR) Stocke les JTI invalides en base de données.

5. Axios Interceptors : api.js

5. Intercepteurs Axios : api.js

api.js is the central Axios instance. It wraps every authenticated HTTP call with two interceptors: the request interceptor injects the Bearer token, and the response interceptor handles 401 errors by attempting an automatic token refresh.

api.js est l'instance Axios centrale. Elle enveloppe chaque appel HTTP authentifié avec deux intercepteurs : l'intercepteur de requête injecte le Bearer token, et l'intercepteur de réponse gère les erreurs 401 en tentant un rafraîchissement automatique du token.

Request interceptor : Bearer injection / Injection du Bearer

```
// api.js - request interceptor
api.interceptors.request.use((config) => {
  // Do NOT inject token on auth routes themselves
  const isAuthRoute =
    config.url?.includes("token/refresh") ||
    config.url?.includes("login") ||
    config.url?.includes("register");

  if (!isAuthRoute) {
    const raw = localStorage.getItem(SESSION_KEY);
    if (raw) {
      const session = JSON.parse(raw);
      if (session?.access) {
        config.headers.Authorization = `Bearer ${session.access}`;
      }
    }
  }
  return config;
});
```

What it does	Reads access token from localStorage and adds it to every non-auth request header as: Authorization: Bearer <token>. Lit le token depuis localStorage et l'ajoute à chaque en-tête de requête non-auth.
isAuthRoute guard	Prevents injection on login, register, token/refresh — these don't need a token and injecting could cause issues. Empêche l'injection sur login, register, token/refresh.
Why localStorage?	The interceptor reads the current token from localStorage on every request, so it automatically picks up a refreshed token without needing a callback. Lit le token courant à chaque requête, récupérant automatiquement un token rafraîchi.

Response interceptor : Auto-refresh on 401 / Auto-refresh sur 401

```
// api.js - response interceptor (simplified)
api.interceptors.response.use(
  (response) => {
    // Auto-unwrap DRF pagination: { count, results: [...] } → [...]
    if (response.data?.results && "count" in response.data) {
      response.data = response.data.results;
    }
    return response;
  },
  async (error) => {
    const original = error.config;
    const is401 = error.response?.status === 401;
    const alreadyRetried = original._retry;
    const isAuthRoute = /* login | register | token/refresh | logout */ false;

    if (is401 && !alreadyRetried && !isAuthRoute) {
      original._retry = true; // prevent infinite retry loop

      try {
        const session = JSON.parse(localStorage.getItem(SESSION_KEY));

        // Call token/refresh using plain axios (not api instance) to avoid loop
        const { data } = await axios.post(`${BASE_URL}/auth/token/refresh/`, {
          refresh: session.refresh,
        });

        // Update localStorage with new tokens (refresh rotates too)
        localStorage.setItem(SESSION_KEY, JSON.stringify({
          ...session,
          access: data.access,
          refresh: data.refresh ?? session.refresh,
        }));

        // Retry original request with new access token
        original.headers.Authorization = `Bearer ${data.access}`;
        return api(original);
      } catch (_refreshError) {
        // Refresh failed (token expired/blacklisted) → force logout
        localStorage.removeItem(SESSION_KEY);
        window.location.href = "/login";
        return Promise.reject(_refreshError);
      }
    }
    return Promise.reject(error);
  }
);
```

Auto-refresh flow step by step / Flux auto-refresh étape par étape

#	Step / Étape	Details EN / FR
1	API call returns 401	(EN) The access token has expired (60-min TTL). Server returns HTTP 401 Unauthorized. (FR) Le token d'accès a expiré (TTL 60 min). Le serveur retourne HTTP 401 Unauthorized.
2	Interceptor catches it	(EN) is401=true, alreadyRetried=false, isAuthRoute=false → enters refresh logic. (FR) is401=true, alreadyRetried=false, isAuthRoute=false → entre dans la logique de refresh.
3	Set _retry=true	(EN) Guards against infinite loops: if the retry also returns 401, the interceptor won't run again. (FR) Protège contre les boucles infinies : si le retry retourne aussi 401, l'intercepteur ne relance pas.
4	POST /auth/token/refresh/	(EN) Sends refresh token to Django SimpleJWT. Uses plain axios (not api instance) to avoid triggering the interceptor again. (FR) Envoie le refresh token à Django SimpleJWT. Utilise axios brut pour éviter de déclencher l'intercepteur.
5a	Refresh succeeds	(EN) New access (and possibly refresh) token received. localStorage updated. Original request retried with new token. (FR) Nouveau access token reçu. localStorage mis à jour. Requête originale rejouée avec le nouveau token.
5b	Refresh fails	(EN) Refresh token expired or blacklisted. Session cleared from localStorage. User redirected to /login. (FR) Refresh token expiré ou blacklisté. Session effacée du localStorage. Utilisateur redirigé vers /login.

6. AuthContext & useAuth hook

6. AuthContext & hook useAuth

AuthContext.jsx is the single source of truth for authentication state in the React component tree. It wraps the service calls and exposes a clean API to components via the useAuth() hook.

AuthContext.jsx est la source unique de vérité pour l'état d'authentification dans l'arbre de composants React. Il enveloppe les appels de service et expose une API claire aux composants via le hook useAuth().

Context value exposed / Valeur de contexte exposée

user	(EN) The authenticated user object: { id, email, first_name, last_name, company_name, role } or null. (FR) L'objet utilisateur connecté ou null.
token	(EN) Current JWT access token string or null. (FR) Chaîne du token JWT d'accès courant ou null.
ready	(EN) Boolean, true once the startup localStorage check is complete. Used to prevent route flicker. (FR) Boolean, true une fois la vérification localStorage initiale terminée. Évite le clignotement des routes.
isAuthenticated	(EN) Computed: Boolean(user). true if any user is logged in. (FR) Calculé : Boolean(user). true si un utilisateur est connecté.
login(email, pass)	(EN) Async calls authService.login(), updates user + token state. (FR) Async appelle authService.login(), met à jour user + token.
register(form)	(EN) Async calls authService.register(), updates user + token state. (FR) Async appelle authService.register(), met à jour user + token.
logout()	(EN) Async calls authService.logout(), resets user + token to null. Async appelle authService.logout(), réinitialise user + token à null.
hasRole(role)	(EN) Predicate: user?.role === role. E.g. hasRole('admin') → true/false. (FR) Prédicat : user?.role === role. Ex. hasRole('admin') → true/false.

Source code : AuthContext.jsx

```
// AuthContext.jsx - state + provider
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(null);
  const [ready, setReady] = useState(false); // startup gate

  // Restore session from localStorage on app mount
  useEffect(() => {
    const session = authService.getSession();
    if (session) { setUser(session.user); setToken(session.access); }
    setReady(true);
  }, []);
```

```
const login    = useCallback(async (email, password) => {
  const session = await authService.login(email, password);
  setUser(session.user); setToken(session.access);
  return session;
}, []);

const register = useCallback(async (form) => {
  const session = await authService.register(form);
  setUser(session.user); setToken(session.access);
  return session;
}, []);

const logout   = useCallback(async () => {
  await authService.logout();
  setUser(null); setToken(null);
}, []);

const hasRole  = useCallback((role) => Boolean(user) && user.role === role,
[user]);

return (
  <AuthContext.Provider value={{ user, token, ready, isAuthenticated:
Boolean(user),
                                login, register, logout, hasRole }}>
    {children}
  </AuthContext.Provider>
);
}
```

useAuth hook : usage in components / Utilisation dans les composants

```
// src/hooks/useAuth.js
export function useAuth() {
  const context = useContext(AuthContext);
  if (context === null) {
    throw new Error("useAuth must be used in an <AuthProvider>.");
  }
  return context;
}

// Example usage in any component:
function Dashboard() {
  const { user, logout, hasRole } = useAuth();

  return (
    <div>
      <p>Welcome, {user.first_name}!</p>
      {hasRole("admin") && <AdminPanel />}
      <button onClick={logout}>Sign out</button>
    </div>
  );
}
```

7. Django JWT Configuration

7. Configuration JWT côté Django

The backend uses Django REST Framework + `django-rest-framework-simplejwt` for JWT authentication. Key settings control token lifetimes, rotation, and blacklisting.

Le backend utilise Django REST Framework + `django-rest-framework-simplejwt` pour l'authentification JWT. Les paramètres clés contrôlent les durées de vie des tokens, la rotation et le blacklisting.

SIMPLE_JWT settings : config/settings.py

```
# config/settings.py - SIMPLE_JWT configuration
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=60), # access token: 1 hour
    "REFRESH_TOKEN_LIFETIME": timedelta(days=60),   # refresh token: 60 days
    "ROTATE_REFRESH_TOKENS": True,                  # new refresh token on every /token/refresh/
    call
    "BLACKLIST_AFTER_ROTATION": True, # old refresh token blacklisted immediately
    "AUTH_HEADER_TYPES": ("Bearer",), # Authorization: Bearer <token>
    "USER_ID_FIELD": "id",             # custom User model uses UUID as primary key
    "USER_ID_CLAIM": "user_id",        # JWT payload field name
}

# Installed apps: token_blacklist enables server-side invalidation
INSTALLED_APPS = [
    ...
    "rest_framework_simplejwt",
    "rest_framework_simplejwt.token_blacklist", # stores blacklisted JTIs in DB
    ...
]
```

Django endpoints / Endpoints Django

Method	URL	Action EN / FR
POST	/api/auth/register/	(EN) Create account + return JWT tokens (FR) Créer un compte + retourner les tokens JWT
POST	/api/auth/login/	(EN) Validate credentials + return JWT tokens (FR) Valider les identifiants + retourner les tokens JWT
POST	/api/auth/logout/	(EN) Blacklist refresh token on server. (FR) Blacklister le refresh token sur le serveur
POST	/api/auth/token/refresh/	(EN) Exchange refresh token for new access token (FR) Échanger le refresh token contre un nouveau access token
GET	/api/auth/me/	(EN) Return current user profile (requires Bearer token) (FR) Retourner le profil courant (nécessite un Bearer token)
POST	/api/auth/change-password/	(EN) Change password after verifying old password. (FR) Changer le mot de passe après vérification de l'ancien

8. Summary

8. Résumé

Concept	Implementation	Details EN / FR
Session storage	localStorage	Key: "cyberaudit:session" → { access, refresh, user } (EN) Persists across browser refreshes and tab reopens. (FR) Persiste même après avoir actualisé le navigateur ou rouvert un onglet.
Token injection	api.js interceptor	(EN) Every request automatically gets: Authorization: Bearer <access> (FR) Chaque requête reçoit automatiquement : Authorization: Bearer <access>
Auto-refresh on 401	api.js interceptor	(EN) Catches 401, calls /token/refresh/, updates localStorage, retries. (FR) Capture le 401, appelle /token/refresh/, met à jour localStorage, rejoue.
Force logout on fail	api.js interceptor	(EN) If refresh fails → removeItem(SESSION_KEY) → window.location = /login. (FR) Si refresh échoue → removeItem + redirection /login
Server blacklist	token_blacklist app	(EN) Logout sends refresh to /auth/logout/ → blacklisted in DB immediately. (FR) Logout envoie refresh à /auth/logout/ → blacklisté en BDD immédiatement.
Token rotation	ROTATE_REFRESH_TOKENS	(EN) Each refresh call rotates the token, old one is blacklisted, new one issued. (FR) Chaque appel refresh fait tourner le token, l'ancien est blacklisté.
React state	AuthContext.jsx	user, token, ready, isAuthenticated, hasRole (EN) globally accessible via useAuth(). (FR) Globalement accessible via useAuth().
Startup restore	getSession()	(EN) useEffect on mount reads localStorage and populates React state before routes render. (FR) useEffect au montage lit localStorage avant le rendu des routes.
Error normalisation	extractErrorMessage()	(EN) DRF can return { detail }, { non_field_errors }, or { field: ['msg'] } normalised to one string. (FR) DRF peut retourner 3 formats normalisé en une seule chaîne.

Design principle: No component ever touches `localStorage` directly. All auth state flows through `authService` → `AuthContext` → `useAuth()`. The Axios interceptor in `api.js` is the only place where Bearer tokens are injected or refreshed.

Principe de conception : *Aucun composant ne touche jamais le `localStorage` directement. Tout l'état d'auth transite par `authService` → `AuthContext` → `useAuth()`. L'intercepteur Axios dans `api.js` est le seul endroit où les tokens Bearer sont injectés ou rafraîchis.*